Shortest Path in a Smart City Network

Dijkstra's Algorithm C++ Implementation

Problem Statement

A. Modify the program to select a different source vertex and recalculate shortest distances.
B. Modify the program to display the complete shortest path to a destination, not only its distance.
C. Modify the program to update an edge weight and recalculate the shortest paths.

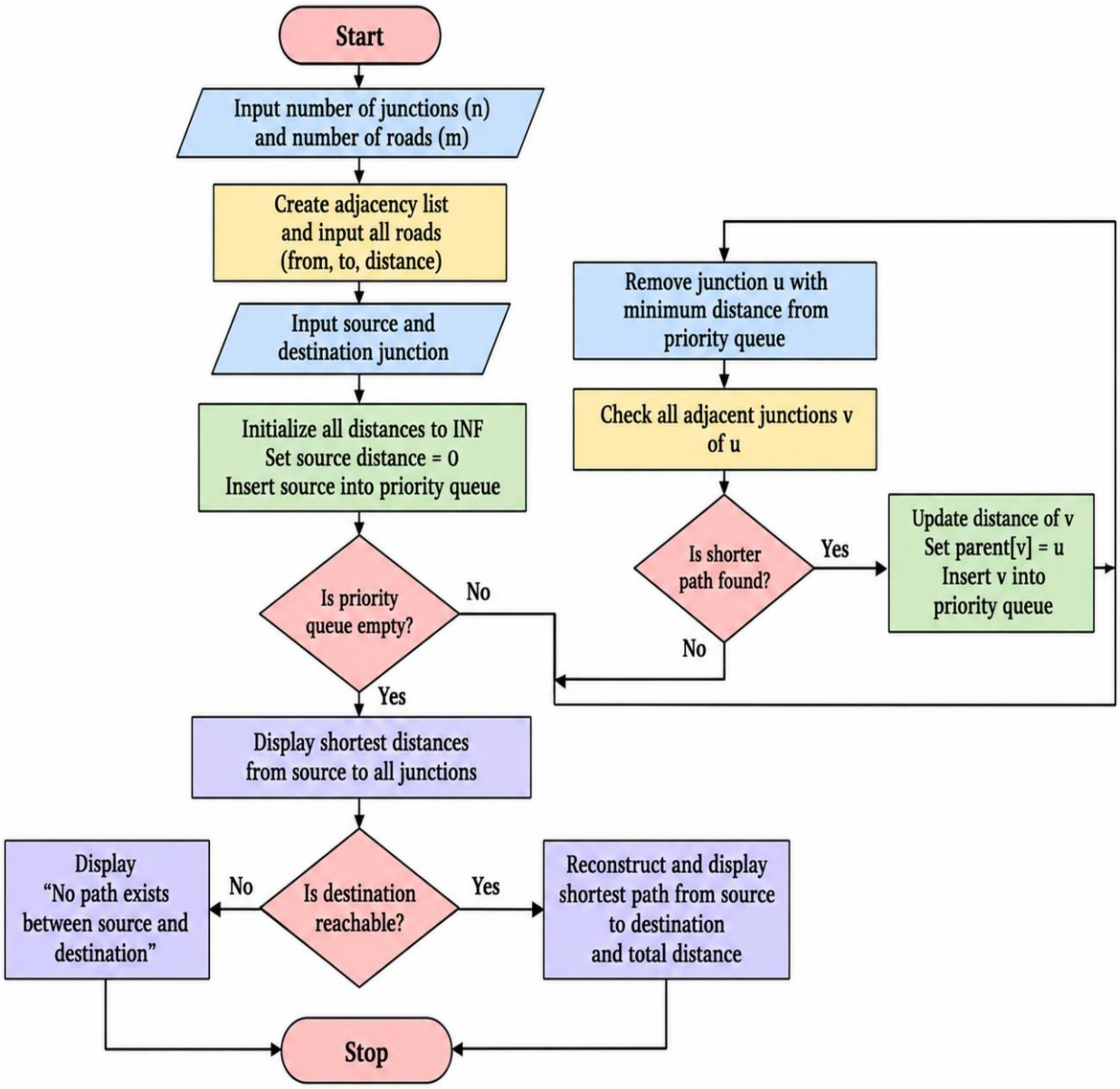
Objectives

- To implement Dijkstra's algorithm using C++.
- To calculate the shortest distance from a source junction to all other reachable junctions.
- To find and display the shortest path between a source and a destination junction.
- To represent a road network using an adjacency list and a priority queue.
- To identify junctions that cannot be reached from the selected source.

Algorithm

1. Start the program.
2. Read the number of junctions n and the number of roads m.
3. Create an adjacency list to represent the road network.
4. Read each road as (from junction, to junction, distance). Add the road in both directions because roads are bidirectional.
5. Read the source junction and the destination junction.
6. Initialize all distances to infinity, set the source distance to zero, and set all parent values to -1.
7. Insert the source junction into a minimum-priority queue.
8. While the priority queue is not empty, remove the junction with the smallest known distance.
9. For each neighboring junction, check whether travelling through the current junction gives a shorter distance.
10. If a shorter path is found, update the distance and parent, then insert the updated distance into the priority queue.
11. When the queue becomes empty, display the shortest distances and mark unreachable junctions.
12. Use the parent values to reconstruct and display the shortest path to the destination and its total distance. If no route exists, display a no-path message.
13. Stop the program.

- **Flowchart**

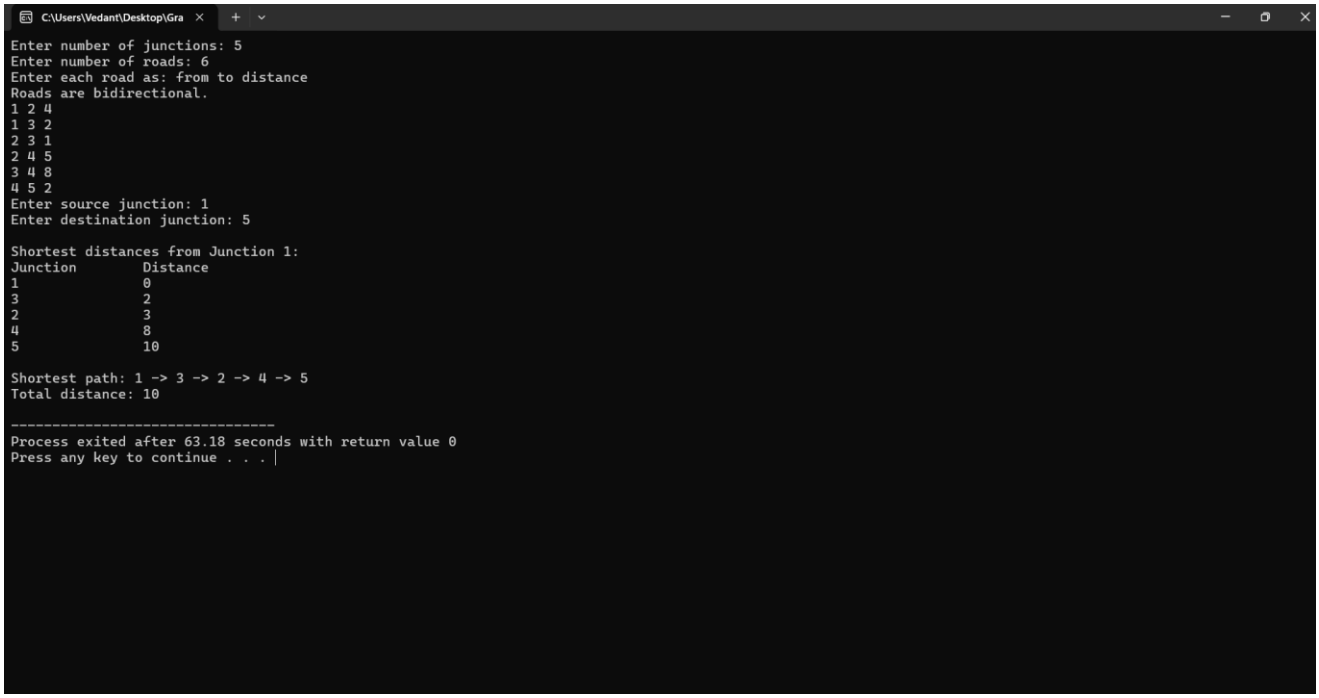

Start
Input number of junctions (n) and number of roads (m)
Create adjacency list and input all roads (from, to, distance)
Input source and destination junction
Initialize all distances to INF
Set source distance = 0
Insert source into priority queue
Is priority queue empty?
No
Yes
Remove junction u with minimum distance from priority queue
Check all adjacent junctions v of u
Is shorter path found?
Yes
No
Update distance of v
Set parent[v] = u
Insert v into priority queue
Display shortest distances from source to all junctions
Is destination reachable?
No
Yes
Display "No path exists between source and destination"
Reconstruct and display shortest path from source to destination and total distance
Stop

- # C++ Code

```
#include <iostream>
#include <vector>
#include <queue>
#include <algorithm>
#include <limits>
#include <functional>
using namespace std;
const long long INF = numeric_limits<long long>::max();
void dijkstra(
    const vector<vector<pair<int, long long> > >& graph,
    int source,
    vector<long long>& dist,
    vector<int>& parent
) {
    int n = (int)graph.size() - 1;
    dist.assign(n + 1, INF);
    parent.assign(n + 1, -1);
    priority_queue<
        pair<long long, int>,
        vector<pair<long long, int> >,
        greater<pair<long long, int> >
    > pq;
    dist[source] = 0;
    pq.push(make_pair(0, source));
    while (!pq.empty()) {
        long long currentDist = pq.top().first;
        int u = pq.top().second;
        pq.pop();
        if (currentDist != dist[u])
            continue;
        for (int i = 0; i < (int)graph[u].size(); ++i) {
            int v = graph[u][i].first;
            long long weight = graph[u][i].second;
            if (dist[u] != INF &&
                dist[u] <= INF - weight &&
                dist[u] + weight < dist[v]) {
                dist[v] = dist[u] + weight;
                parent[v] = u;
                pq.push(make_pair(dist[v], v));
            }
        }
    }
}
void displayDistances(
    const vector<long long>& dist,
    int source
) {
    vector<pair<long long, int> > reachable;
    int n = (int)dist.size() - 1;
    for (int i = 1; i <= n; ++i) {
        if (dist[i] != INF)
            reachable.push_back(make_pair(dist[i], i));
    }
    sort(reachable.begin(), reachable.end());
    cout << "\nShortest distances from Junction "
         << source << ":\n";
    cout << "Junction\tDistance\n";
    for (int i = 0; i < (int)reachable.size(); ++i) {
        cout << reachable[i].second << "\t\t"
             << reachable[i].first << "\n";
    }
    for (int i = 1; i <= n; ++i) {
        if (dist[i] == INF)
            cout << i << "\t\tUnreachable\n";
    }
}
void displayPath(
    const vector<long long>& dist,
    const vector<int>& parent,
    int source,
    int destination
) {
    if (dist[destination] == INF) {
        cout << "\nNo path exists between Junction "
             << source << " and Junction "
             << destination << ".\n";
        return;
    }
    vector<int> path;
    int current = destination;
    while (current != -1) {
        path.push_back(current);
        current = parent[current];
    }
    reverse(path.begin(), path.end());
    cout << "\nShortest path: ";
    for (int i = 0; i < (int)path.size(); ++i) {
```

```
        cout << path[i];
        if (i < (int)path.size() - 1)
            cout << " -> ";
    }
    cout << "\nTotal distance: "
         << dist[destination] << "\n";
}
int main() {
    int n, m;
    cout << "Enter number of junctions: ";
    cin >> n;
    cout << "Enter number of roads: ";
    cin >> m;
    if (n <= 0 || m < 0) {
        cout << "Invalid input.\n";
        return 1;
    }
    vector<vector<pair<int, long long> > > graph(n + 1);
    cout << "Enter each road as: from to distance\n";
    cout << "Roads are bidirectional.\n";
    for (int i = 0; i < m; ++i) {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        if (u < 1 || u > n ||
            v < 1 || v > n || w < 0) {
            cout << "Invalid road input.\n";
            return 1;
        }
        graph[u].push_back(make_pair(v, w));
        graph[v].push_back(make_pair(u, w));
    }
    int source, destination;
    cout << "Enter source junction: ";
    cin >> source;
    cout << "Enter destination junction: ";
    cin >> destination;
    if (source < 1 || source > n ||
        destination < 1 || destination > n) {
        cout << "Invalid source or destination.\n";
        return 1;
    }
    vector<long long> dist;
    vector<int> parent;
    dijkstra(graph, source, dist, parent);
    displayDistances(dist, source);
    displayPath(dist, parent, source, destination);
    return 0;
}
```

- **Code Output**


C:\Users\Vedant\Desktop\Gra
Enter number of junctions: 5
Enter number of roads: 6
Enter each road as: from to distance
Roads are bidirectional.
1 2 4
1 3 2
2 3 1
2 4 5
3 4 8
4 5 2
Enter source junction: 1
Enter destination junction: 5
Shortest distances from Junction 1:
Junction Distance
1 0
3 2
2 3
4 8
5 10
Shortest path: 1 -> 3 -> 2 -> 4 -> 5
Total distance: 10
Process exited after 63.18 seconds with return value 0
Press any key to continue . . .